PROJECT REPORT

Student Management & Enrollment System

1. Introduction

The Student Management & Enrollment System is a full-stack web application developed to manage student enrollment records. It allows users to create, view, update, search, filter, and soft-delete student records, while providing dashboard statistics for total, active, and inactive students.

2. Objectives

- Build a centralized application for managing student enrollment information.
- Expose REST APIs for student management operations.
- Store student information persistently in PostgreSQL.
- Provide search, filtering, and dashboard statistics.
- Apply input validation, duplicate-email prevention, and consistent error handling.
- Verify key application workflows using automated integration tests.

3. Technology Stack

Component	Technology
Programming language	Java 21
Backend	Spring Boot, Spring Data JPA, Hibernate, REST APIs
Database	PostgreSQL
Frontend	HTML, CSS, JavaScript, Fetch API
Build and testing	Gradle, JUnit 5, Spring Boot Test

4. System Architecture

Frontend → **Controller** → **Service** → **Repository** → **PostgreSQL**

The frontend sends HTTP requests to the REST controllers. Controllers validate and route requests to the service layer, where business rules are applied. The repository layer uses Spring Data JPA to interact with PostgreSQL. DTOs separate incoming request data from the database entity.

5. Implemented Features

- Create, list, retrieve, update, and soft-delete student records.
- Search students by partial name or email match.
- Filter students by course and ACTIVE/INACTIVE status.
- Display total, active, and inactive student counts on the dashboard.
- Validate required fields, email format, and exactly 10 phone-number digits.
- Prevent duplicate email addresses through application checks and a database unique constraint.
- Return appropriate HTTP status codes for validation errors, missing students, and duplicate emails.

6. Database Design

PostgreSQL stores student information in a single **students** table. The table includes an auto-generated ID, full name, unique email, phone number, course, enrollment date, status, and a boolean soft-delete indicator. Soft deletion keeps the record in the database while excluding it from normal listings and dashboard counts.

7. Testing

Automated integration tests were implemented using JUnit and Spring Boot Test. The following scenarios were executed successfully:

- Successful student creation (201 Created).
- Rejection of an invalid email address (400 Bad Request).
- Rejection of a duplicate email address (409 Conflict).

- Successful student update and verification.

8. Conclusion

The project demonstrates a complete full-stack workflow using Java, Spring Boot, PostgreSQL, and JavaScript. It provided practical experience in layered backend architecture, REST API development, database integration, validation, exception handling, frontend-backend communication, and automated testing.